15. Stream Hierarchy and Methods — Quick Notes

Top Interface

BaseStream

Provides common methods:

- parallel()
- sequential()
- unordered()

Sub Interfaces

Interface	Used For
IntStream	primitive int
LongStream	primitive long
DoubleStream	primitive double
Stream<T>	non-primitive objects

Important Point

Primitive stream interfaces: avoid boxing/unboxing overhead

Example: IntStream

Stores: int directly.

BaseStream Methods

Method	Purpose
parallel()	Converts to parallel stream
sequential()	Converts to sequential stream
unordered()	Removes ordering

---

parallel()

Enable parallel processing.

Example: list.stream().parallel()

Uses: multiple CPU cores

---

sequential()

Convert back to normal sequential stream.

Example: stream.sequential()

---

unordered()

Removes ordering constraint.

Useful for: better parallel processing

Common Stream Methods
These methods are available in:

- Stream
- IntStream
- LongStream
- DoubleStream

---

filter()
Intermediate Operation
Uses: Predicate
Example: .filter(x -> x % 2 == 0)
Filters: even elements

---

forEach()
Terminal Operation
Performs action on each element.
Example: .forEach(System.out::println)

---

min() / max()
Terminal Operations
Find:
- minimum element
- maximum element

Uses:
- Comparable
- Comparator

Example: stream.min(Integer::compare)

---

skip()
Intermediate Operation
Skips elements.
Example: .skip(5)
Skips: first 5 elements

---

findFirst()
Terminal Operation
Returns: first matching element
Example: .findFirst()

Optional Return Type
`findFirst()` returns: Optional
Because: result may or may not exist

---

count()

Terminal Operation
Counts stream elements.
Example: .count()
Returns: long

---

distinct()
Intermediate Operation
Removes duplicates.
Example: .distinct()

---

sorted()
Intermediate Operation
Returns:
sorted stream
Example: .sorted()

Important Point
Methods like:
- distinct()
- sorted()

modify stream order/content.

---

reduce()
Terminal Operation
Used for: combining elements
Example: .reduce((x,y) -> x + y)

---

reduce() Use Cases

Use Case	Example
Sum	Total marks
String combine	Concatenation
Product	Multiplication

---

collect()
Terminal Operation
Used to convert: Stream → Collection
Example: .collect(Collectors.toList())

Primitive Stream Special Methods
Present only in:
- IntStream
- LongStream
- DoubleStream

---

sum()
Find total sum.
Example: IntStream.of(10,20,30).sum()
Output: 60

average()
Find average value.
Example: IntStream.of(10,20,30).average()

boxed()
Convert primitive stream → object stream.
Example: IntStream.of(10,20).boxed()
Converts: IntStream → Stream<Integer>

Intermediate vs Terminal

Type	Returns
Intermediate	Stream
Terminal	Final Result

Examples

Intermediate	Terminal
filter()	forEach()
map()	count()
distinct()	findFirst()
sorted()	collect()

Important Quick Points

Topic	Point
BaseStream	Root stream interface
Stream<T>	Non-primitive stream
IntStream	Primitive int stream
filter()	Intermediate op
forEach()	Terminal op
count()	Returns long
reduce()	Combines elements
collect()	Stream → Collection
boxed()	Primitive → Wrapper stream